

Python 小程序 2

```
# 以此代码演示了函数作为参数、lambda 表达式、异常处理和字典映射的实际应用
def add(a: float, b: float) -> float:
    return a + b
def subtract(a: float, b: float) -> float:
    return a - b
def multiply(a: float, b: float) -> float:
    return a * b
def divide(a: float, b: float) -> float:
    if b == 0:
        raise ZeroDivisionError("除数不能为零")
    return a / b
def power(a: float, b: float) -> float:
    return a ** b
def modulus(a: float, b: float) -> float:
    return a % b
class Calculator:
    """多功能计算器类"""
    def __init__(self):
        # 操作符映射到对应的函数
        self.operations = {
            '+': add,
            '-': subtract,
            '*': multiply,
            '/': divide,
            '**': power,
            '%': modulus,
            '^': power
        }
        self.history = []
    def calculate(self, expression: str) -> float:
        """计算表达式"""
        # 分割表达式
        for op in self.operations.keys():
            if op in expression:
                parts = expression.split(op)
                if len(parts) != 2:
                    raise ValueError("表达式格式错误")
```

```

        try:
            a = float(parts[0].strip())
            b = float(parts[1].strip())
            result = self.operations[op](a, b)
            # 记录历史
            self.history.append(f"{expression} = {result:.4f}")
            return result
        except ValueError:
            raise ValueError("数字格式错误")
    raise ValueError("不支持的运算符")

def show_history(self) -> None:
    """显示计算历史"""
    if not self.history:
        print("暂无计算历史！")
        return
    print("\n" + "="*50)
    print("计算历史:")
    for i, record in enumerate(self.history, 1):
        print(f"{i}. {record}")
    print("="*50)

def run(self) -> None:
    """运行计算器"""
    print("==== 多功能科学计算器 =====")
    print("支持的运算符: +, -, *, /, **(幂), %(取余), ^(幂)")
    print("输入格式: 数字 运算符 数字 (例如: 5 + 3)")
    print("输入 'history' 查看历史, 输入 'clear' 清空历史, 输入 'exit' 退出")
    while True:
        expression = input("\n 请输入计算表达式: ").strip()
        if expression.lower() == 'exit':
            print("感谢使用, 再见！")
            break
        elif expression.lower() == 'history':
            self.show_history()
            continue
        elif expression.lower() == 'clear':
            self.history.clear()
            print("计算历史已清空！")
            continue
    try:

```

```
        result = self.calculate(expression)
        print(f"计算结果: {result:.4f}")
    except Exception as e:
        print(f"计算错误: {e}")

if __name__ == "__main__":
    calc = Calculator()
    calc.run()
```